

Day 30 – Docker Images & Container Lifecycle (Step-by-Step Guide)

Task 1: Docker Images

Step 1: Pull Images from Docker Hub

None

```
docker pull nginx
docker pull ubuntu
docker pull alpine
```

Step 2: List All Images

None

```
docker images
```

Example Output:

None

REPOSITORY	TAG	IMAGE ID	SIZE
nginx	latest	abc123	192MB
ubuntu	latest	def456	79MB
alpine	latest	ghi789	8MB

Why is Alpine Smaller?

Ubuntu	Alpine
General-purpose OS	Minimal Linux distro
Includes many packages	Only essential packages
Larger size	Very lightweight
Good for development	Good for containers

Note: Alpine uses BusyBox and musl libc, making it extremely small.

Step 3: Inspect an Image

None

```
docker image inspect nginx
```

You can see:

- Image ID
 - Creation Date
 - Architecture
 - Environment Variables
 - Entrypoint
 - Layers
 - Size
-

Step 4: Remove an Image

None

```
docker rmi ubuntu
```

If image is used by a container:

None

```
docker rmi -f ubuntu
```

Task 2: Docker Image Layers

View Image History

None

```
docker image history nginx
```

Example:

None

IMAGE	CREATED	CREATED BY	SIZE
abc123	2 weeks ago	CMD ["nginx"]	0B
xyz456	2 weeks ago	RUN apt update	25MB

uvw789

2 weeks ago

COPY files

10MB

What are Layers?

Docker images are built in layers.

Example Dockerfile:

None

```
FROM ubuntu
RUN apt update
RUN apt install nginx -y
COPY . /app
```

Layers:

None

```
Layer 1 → Ubuntu Base Image
Layer 2 → apt update
Layer 3 → nginx install
Layer 4 → copy application files
```

Why Docker Uses Layers?

- Faster builds
- Reuse existing layers
- Saves disk space
- Easy image distribution

Caching Example

First build:

None

```
docker build -t myapp .
```

Second build:

None

```
docker build -t myapp .
```

Docker reuses unchanged layers from cache.

Task 3: Container Lifecycle

Create Container (Not Started)

None

```
docker create --name test-container ubuntu
```

Check status:

None

```
docker ps -a
```

State:

None

```
Created
```

Start Container

None

```
docker start test-container
```

Check:

None

```
docker ps -a
```

State:

None

Running

Pause Container

None

`docker pause test-container`

Check:

None

`docker ps -a`

State:

None

Paused

Unpause Container

None

`docker unpause test-container`

State:

None

Running

Stop Container

None

`docker stop test-container`

State:

None

Exited

Restart Container

None

`docker restart test-container`

State:

None

Running

Kill Container

None

`docker kill test-container`

State:

None

Exited

Remove Container

None

`docker rm test-container`

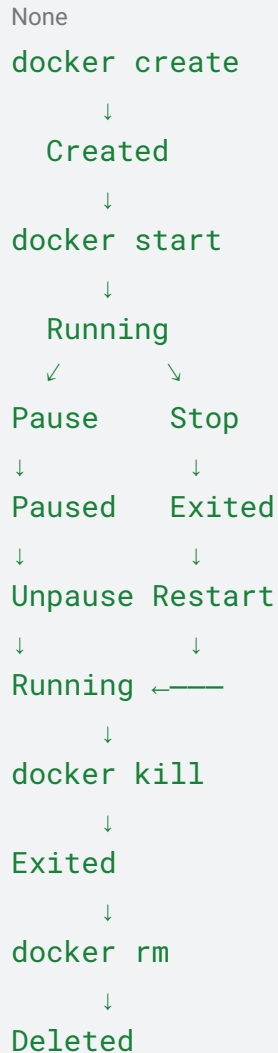
Verify:

None

`docker ps -a`

Container should disappear.

Container Lifecycle Diagram



Task 4: Working with Running Containers

Run Nginx Container in Detached Mode

None

```
docker run -d --name my-nginx -p 8080:80 nginx
```

Verify:

None

```
docker ps
```

Open Browser:

None

```
http://<server-ip>:8080
```

View Logs

None

```
docker logs my-nginx
```

Follow Logs in Real Time

None

```
docker logs -f my-nginx
```

Exit:

None

```
Ctrl + C
```

Enter Running Container

None

```
docker exec -it my-nginx bash
```

If bash unavailable:

None

```
docker exec -it my-nginx sh
```


Explore:

```
None
pwd
ls
cd /etc/nginx
ls
```

Exit:

```
None
exit
```

Run Single Command Without Entering

```
None
docker exec my-nginx ls /
```

Another example:

```
None
docker exec my-nginx cat /etc/os-release
```

Inspect Container

```
None
docker inspect my-nginx
```

Find:

IP Address

```
None
"IPAddress": "172.17.0.2"
```

Port Mapping

None

```
"HostPort": "8080"
```

Mounts

None

```
"Mounts": []
```

Task 5: Cleanup

Stop All Running Containers

None

```
docker stop $(docker ps -q)
```

Remove All Stopped Containers

None

```
docker rm $(docker ps -aq)
```

Remove Unused Images

None

```
docker image prune -a
```

Check Docker Disk Usage

None

```
docker system df
```

Example:

None

TYPE	TOTAL	ACTIVE	SIZE
Images	5	2	1.2GB
Containers	3	1	50MB
Local Volumes	2	2	100MB

Complete Cleanup

None

```
docker system prune -a
```

Remove everything unused:

- Stopped containers
- Unused networks
- Dangling images
- Build cache

Commands Summary

None

```
docker pull nginx
docker images
docker image inspect nginx
docker image history nginx

docker create --name test ubuntu
docker start test
docker pause test
docker unpause test
docker stop test
docker restart test
docker kill test
docker rm test

docker run -d --name my-nginx -p 8080:80 nginx
docker logs my-nginx
```

```
docker logs -f my-nginx  
docker exec -it my-nginx bash  
docker inspect my-nginx
```

```
docker stop $(docker ps -q)  
docker rm $(docker ps -aq)  
docker image prune -a  
docker system df  
docker system prune -a
```